CIS 1921 — FINAL PROJECT

Constraint-Based Training, Nutrition & Health Schedule Optimizer

A weekly fitness + nutrition optimizer with four directly comparable LP/MIP & CP-SAT formulations.

Aryan Jeena · Aadithya Srinivasan

April 28, 2026

Real students don’t plan one dimension at a time.

Training adherence depends on more than macros. WHEN you eat, WHEN you lift, and WHEN you sleep all matter — and real students juggle class schedules, dining-hall windows, budgets, and recovery rules at the same time.

The standard “meal planner + calendar app” split

Solves each dimension independently and produces plans that are technically compliant but practically useless — e.g. a meal prep that finishes 15 minutes before a heavy squat session, or a schedule that hits macros only by ignoring the dining-hall window.

→ The joint formulation is the whole point.

Time grid, decision variables, hard + soft constraints.

DOMAIN

7 days × 48 half-hour slots = 336 slots/week

Everything in integers (cents, grams, slots). CP-SAT requires integer coefficients and integer arithmetic also keeps the unit tests reproducible across runs.

DECISION VARIABLES (joint model)

- serve[d, m, f] — int servings of food f in meal m on day d
- meal_active[d, m] — boolean, reified to total servings ≥ 1
- meal_start[d, m] + meal_iv[d, m] — optional CP-SAT interval
- wk_sched[d, w], wk_start[d, w], wk_iv[d, w] — workout placement
- sleep block per night + per-slot hydration booleans

SHARED SOLVER CONTRACT

All four solvers implement BaseSolver

Inputs

- UserProfile — macros, budget, exclusions, availability, pantry
- list[FoodItem] — Penn Dining + USDA + curated sample
- list[WorkoutTemplate] — duration, intensity, recovery rules

Output

- SolverResult { status, objective, runtime_s, plan }
- Plan with daily meals, workouts, sleep blocks, totals

→ Same input/output across all formulations means the experiment runner produces apples-to-apples runtime, feasibility, and objective comparisons.

Four directly comparable formulations.

	nutrition_only	two_stage	joint_cpsat	joint_warmstart
Macros + budget	✓	✓	✓	✓
Workout scheduling	✗	✓	✓	✓
Sleep + recovery spacing	✗	✓	✓	✓
Peri-workout meal timing	✗	partial	✓	✓
Trade macro slack ↔ schedule	✗	✗	✓	✓
Warm-start from two-stage	✗	✗	✗	✓
Technique	LP / MIP	MIP → CP-SAT	CP-SAT	CP-SAT + AddHint

Joint CP-SAT is the marquee model — a single CP-SAT model where the solver can trade a small macro deviation for a better-timed schedule, which neither baseline can do.

Hard constraints vs soft objective, kept explicit in code.

HARD CONSTRAINTS (the user's goals)

Encoded as model constraints

- Daily calorie band ± tolerance
- Daily protein floor
- Weekly budget cap
- Dietary exclusions / pantry restriction
- AddNoOverlap on meals + workouts
- Recovery spacing between hard workouts
- Max consecutive hard days
- Workout count range (min, max)
- Sleep + availability windows

SOFT OBJECTIVE (9 weighted terms)

Linearised cost minimised by every solver

- Calorie / carb / fat deviation penalties
- Cost (per cent of food)
- Per-meal protein shortfall
- Peri-workout meal-timing miss
- Hydration shortfall
- Preferred / avoided workout days
- Convenience bonus (negative term)

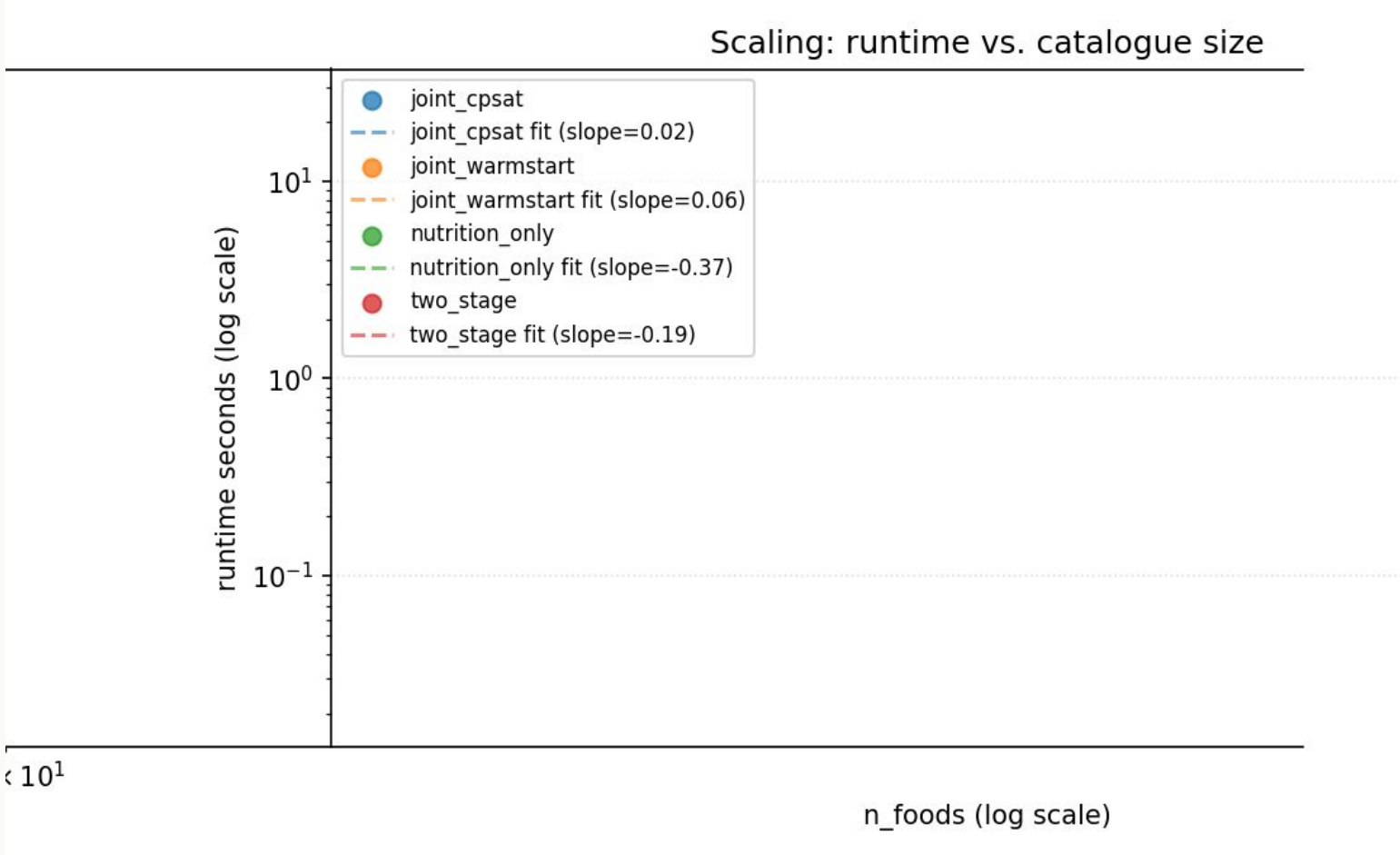
Adherence is a CONSTRAINT. Cheapness, macro fit, and schedule pleasantness are the OBJECTIVE.

Switching to the terminal — all four solvers, balanced scenario.

```
$ python scripts/run_demo.py

Demo run on scenario='balanced_1921' (56 foods, 10 workouts)

--- nutrition_only ---  status=OPTIMAL  runtime=0.09s   obj=-266
--- two_stage      ---  status=OPTIMAL  runtime=0.21s   obj=   0
--- joint_cpsat    ---  status=OPTIMAL  runtime=1.33s   obj=-266
--- joint_warmstart---  status=OPTIMAL  runtime=1.52s   obj=-266

$ python -m src.app.cli solve --preset lean_bulk --solver joint_cpsat --figure
```

Live: a 7-day plan rendered with meals in their availability windows, 3 workouts respecting recovery rules, sleep blocks each night, and 3120 kcal / 202g protein / day matching the lean-bulk targets.

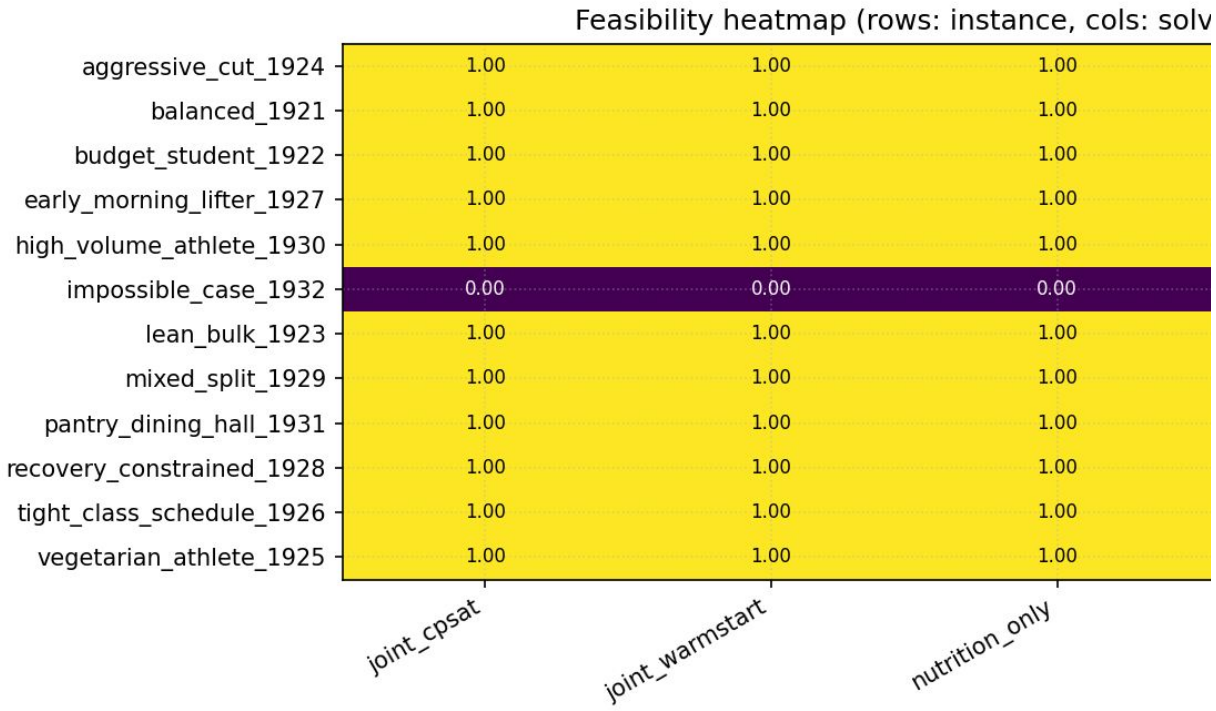
Runtime scaling — catalog size 10 → 200 foods.



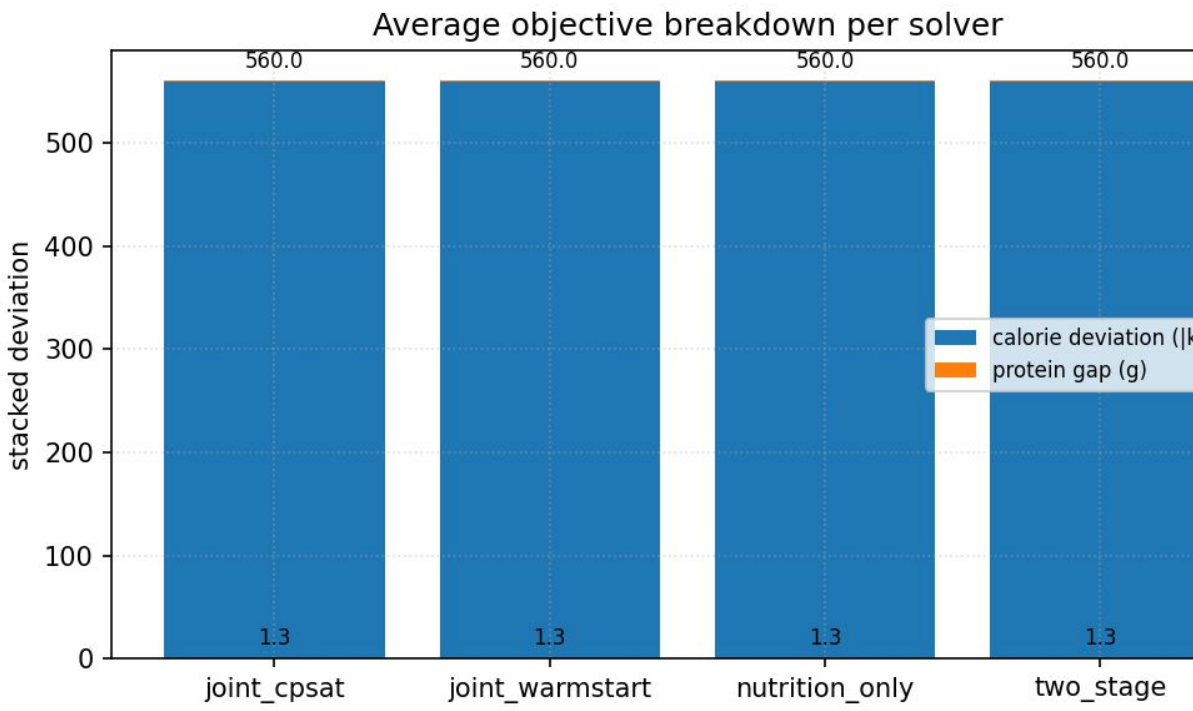
WHAT THIS SHOWS

- nutrition_only stays sub-second across the entire sweep — it's a textbook diet MIP.
- Joint CP-SAT remains practical up to ~80 foods. Past that the (food × meal × day) combinatorics start to bite.
- Warm-start narrows the gap on big catalogs by seeding the joint model from a feasible two-stage plan via model.AddHint(...).

Feasibility heatmap and objective breakdown.



Feasibility per (instance × solver)



Stacked objective by penalty term

Where the joint pays off: instances where the two-stage commits to food choices the scheduler then cannot fit. Where it doesn't: small loose instances where the MIP-then-schedule baseline reaches OPTIMAL faster than the joint can presolve.

What we learned.

Joint > two-stage when constraints are tight

The two-stage decomposition can commit to food choices the scheduler then cannot fit. The joint model trades a small macro deviation for a feasible schedule directly — exactly the gap CLAUDE.md predicted.

Warm-start helps where it matters

On large catalogs the joint solver is faster from a seeded plan via model.AddHint(...). On tiny loose instances the two-stage pre-pass is wasted overhead, so the cold joint wins.

Assumption-literal IIS pays off

When the joint model is infeasible, SufficientAssumptionsForInfeasibility() returns the smallest group(s) the user must relax: calorie band, protein floor, budget, workout count, or scheduling overlaps.

Hydration was free

Sliding-window pairwise spacing on a daytime window with a soft shortfall term — never increased runtime past the time limit on any feasible instance.

Future work, repo, thanks.

FUTURE WORK

Solver scaling

Column generation / Benders-style decomposition past 200 foods.

Online re-optimization

Missed workout triggers a partial re-solve over remaining days.

Real Penn Dining scrape

Cookie-aware Playwright fetcher behind a feature flag.

Hard per-meal protein floor

Currently soft; promote in pantry mode for athlete users.

WHAT'S IN THE BOX

- 5,176 LOC src/ (≈ 2,000 LOC pure solver code)
- 37 unit tests, all passing
- Parameterized instance generator (10 → 200 foods)
- 5 figure types × 3 sweep prefixes
- Penn Dining + USDA + curated sample data
- Streamlit UI + CLI + JSON presets
- Final report — 456 lines of analysis

Thank you — questions?

github.com/aryan-jeena/CIS1921Project · Aryan Jeena & Aadithya Srinivasan